

Universidad de El Salvador

Facultad de Ingeniería y Arquitectura

Escuela de Sistemas Informáticos

Curso de Especialización en Machine Learning

Laboratorio 2

Question Answering y Retrieval-Augmented Generation

sobre documentación académica de la Universidad de El Salvador

Informe académico final

Estudiantes: Josias Abner Rivas Fuentes
Elmer Edenilson Rosales Molina

Catedrático: Vladimir Dias

Fecha: 14 de agosto de 2026

github.com/abner-rivas/Laboratorio—ML

Índice

2	Introducción	1
3	Objetivos	1
3.1	Objetivo general	1
3.2	Objetivos específicos	1
4	Fundamentos teóricos	1
4.1	Procesamiento de Lenguaje Natural	1
4.2	Question Answering	2
4.3	QA extractivo	2
4.4	Arquitectura Transformer	2
4.5	Tokenización	2
4.6	Modelos preentrenados	2
4.7	Score de confianza	2
4.8	top_k	2
4.9	Chunking	3
4.10	Overlap	3
4.11	Embeddings	3
4.12	Búsqueda semántica	3
4.13	FAISS	3
4.14	RAG	3
5	Herramientas y tecnologías	3
6	Desarrollo de Question Answering	4
6.1	Instalación y configuración	4
6.2	Carga del modelo	4
6.3	Tokenizer	4
6.4	Pipeline QA	4
6.5	Función de respuesta	4
6.6	Interpretación del resultado	5
7	Experimentos iniciales	5
7.1	Contexto del Sistema Solar	5
7.2	Preguntas de prueba	5
7.3	Análisis de respuestas	5
7.4	Análisis de score	5
7.5	Uso de top_k	5
7.6	Preguntas cuya respuesta no está en el contexto	6
8	Actividad de laboratorio: contexto propio	6
8.1	Descripción del contexto seleccionado	6
8.2	Contexto de aproximadamente 300 palabras	6
8.3	Diseño de las 10 preguntas	7
8.4	Resultados obtenidos	7

8.5	Tabla de respuestas y scores	7
8.6	Análisis de errores	7
8.7	Reflexión	8
9	Comparación de modelos QA	8
9.1	Modelos utilizados	8
9.2	Contexto de evaluación	8
9.3	Preguntas	8
9.4	Resultados por modelo	8
9.5	Score	8
9.6	Tiempo de inferencia	9
9.7	Ventajas y desventajas	9
9.8	Análisis comparativo	9
9.9	Conclusiones	9
10	Experimentos con diferentes contextos	9
10.1	Bases de Datos / IA / Redes Neuronales / Ciencia de Datos	9
10.2	Resultados	10
10.3	Artículo de Wikipedia	10
10.4	Cinco preguntas	10
10.5	Comparación de los tres modelos	10
11	Question Answering sobre documentos extensos	10
11.1	Problema de longitud de los Transformers	10
11.2	Estrategia de chunking	10
11.3	Tamaño de fragmentos	11
11.4	Implementación	11
11.5	Selección de la mejor respuesta	11
11.6	Análisis de resultados	11
12	Evaluación del overlap	11
12.1	Chunking sin overlap	11
12.2	Chunking con overlap	11
12.3	Resultados	11
12.4	Comparación	12
12.5	Conclusiones	12
13	Sistema RAG	12
13.1	Arquitectura propuesta	12
13.2	Generación de embeddings	13
13.3	Modelo de embeddings	13
13.4	Construcción de la base vectorial	13
13.5	FAISS	13
13.6	Búsqueda semántica	13
13.7	Recuperación Top-K	13
13.8	Question Answering sobre fragmentos recuperados	13
13.9	Extensión multidocumento	14

14	Proyecto final	15
14.1	Descripción del problema	15
14.2	Documento seleccionado	15
14.3	Arquitectura del sistema	15
14.4	Procesamiento del documento	15
14.5	Embeddings	15
14.6	FAISS	16
14.7	Recuperación semántica	16
14.8	QA	16
14.9	Preguntas de prueba	16
14.10	Resultados	16
14.11	Análisis de errores	16
14.12	Limitaciones	17
14.13	Mejoras propuestas	17
15	Experimentos adicionales	17
15.1	Modelo de embeddings alternativo	17
15.2	Tamaño de chunk: 300, 500 y 800	18
15.3	Diferentes valores de overlap	18
15.4	Diferentes valores de k	18
15.5	Impacto del tamaño de la base vectorial	18
16	Conclusiones	18
17	Referencias bibliográficas	19
18	Anexos	21
18.1	Preguntas del contexto propio	21
18.2	Preguntas monodocumento	21
18.3	Protocolos de comparación de modelos	21
18.4	Comparación descriptiva mono y multidocumento	22
18.5	Evidencia y repositorio	22

2. Introducción

Question Answering (QA) busca entregar una respuesta concreta a una pregunta formulada en lenguaje natural a partir de un contexto disponible. En QA extractivo la salida es un fragmento del texto, no una redacción nueva. Esta tarea aplica técnicas de Procesamiento de Lenguaje Natural (PLN) para reducir el esfuerzo de localizar información puntual, pero su utilidad depende de que el contexto contenga evidencia pertinente.

El laboratorio desarrolló de forma progresiva un sistema QA en español: partió de contextos breves, comparó tres modelos BETO, procesó un documento institucional extenso y añadió recuperación semántica con embeddings y FAISS. El objetivo general fue estudiar cómo la fragmentación, la recuperación Top-K y el extractor afectan la respuesta final. La solución se extendió luego a nueve documentos de la Universidad de El Salvador (UES), conservando la procedencia de cada fragmento.

3. Objetivos

3.1 Objetivo general

Diseñar y evaluar un sistema de preguntas y respuestas extractivo en español que pueda operar sobre contextos breves y documentación institucional extensa, incorporando recuperación semántica para seleccionar evidencia antes de extraer la respuesta.

3.2 Objetivos específicos

- Implementar e interpretar un flujo QA extractivo con modelos preentrenados en español.
- Comparar tres checkpoints BETO con preguntas y criterios comunes.
- Evaluar tamaño de chunk, overlap, Top-K, modelo de embeddings y tamaño de la base vectorial.
- Construir un índice FAISS trazable y medir por separado recuperación y exactitud de respuesta.
- Aplicar la configuración seleccionada a uno y varios documentos institucionales, identificando errores y limitaciones.

4. Fundamentos teóricos

4.1 Procesamiento de Lenguaje Natural

El PLN reúne métodos para representar y analizar lenguaje humano mediante sistemas computacionales. En este trabajo permite transformar preguntas y textos en tokens y vectores que los modelos pueden procesar.

4.2 Question Answering

QA recibe una pregunta y uno o más contextos para localizar información que la responda. La búsqueda de documentos y QA son etapas distintas: la primera recupera material potencialmente relevante; la segunda identifica la respuesta concreta dentro de ese material.

4.3 QA extractivo

Un modelo extractivo estima las posiciones de inicio y fin de un span en el contexto. Por tanto, solo puede devolver texto presente en la entrada y puede producir un span espurio cuando la evidencia no existe. BERT permite este ajuste mediante dos distribuciones sobre los tokens [2].

4.4 Arquitectura Transformer

Transformer usa atención para relacionar tokens de una secuencia sin recurrencia [1]. BERT emplea encoders Transformer bidireccionales: cada representación incorpora el contexto a ambos lados del token, propiedad útil para delimitar respuestas [2].

4.5 Tokenización

La tokenización convierte pregunta y contexto en identificadores de subpalabras, añade tokens especiales y produce máscaras de atención. El límite conjunto de 512 tokens de los modelos evaluados impide introducir directamente un documento completo y motiva el chunking.

4.6 Modelos preentrenados

Un modelo preentrenado aprende representaciones generales y luego se ajusta a una tarea. BETO es una adaptación de BERT preentrenada para español [3]. En el laboratorio se usaron checkpoints ya ajustados para QA; no se entrenaron pesos desde cero.

4.7 Score de confianza

El score QA expresa la confianza relativa del extractor en un span del contexto recibido. No verifica la pertinencia del contexto y no equivale a Accuracy. En el análisis se consideró alto desde 0.75, medio desde 0.40 y bajo por debajo de 0.40; son umbrales operativos, no universales.

4.8 top_k

En la función QA, top_k permite observar varios spans candidatos ordenados por score. En recuperación, Top-K designa los K chunks semánticos enviados al extractor. Aumentar K amplía cobertura y coste, pero no garantiza que el primer candidato sea correcto.

4.9 Chunking

Chunking divide un texto extenso en fragmentos compatibles con el modelo. Un chunk pequeño ofrece precisión local pero puede separar evidencia; uno grande preserva contexto, aunque incorpora más información irrelevante y se acerca al límite de tokens.

4.10 Overlap

El overlap repite caracteres entre chunks consecutivos. Puede proteger evidencia situada en un corte, a costa de duplicar contenido y aumentar el número de fragmentos e inferencias. Su efecto debe medirse en cada corpus.

4.11 Embeddings

Un embedding es un vector denso que representa el significado aproximado de un texto. Sentence-BERT permite comparar oraciones y párrafos en un espacio semántico [4]. El modelo final basado en MPNet [5] produce vectores de 768 dimensiones.

4.12 Búsqueda semántica

La pregunta y los chunks se codifican en el mismo espacio vectorial. Con vectores normalizados, el producto interno coincide con la similitud coseno:

$$\hat{\mathbf{q}}^T \hat{\mathbf{c}} = \cos(\hat{\mathbf{q}}, \hat{\mathbf{c}}).$$

Esta similitud ordena afinidad temática; no representa confianza QA ni corrección de respuesta.

4.13 FAISS

FAISS implementa búsqueda eficiente sobre vectores [6]. Se usó `IndexFlatIP`, que calcula producto interno exacto. Como los embeddings se normalizan, el ranking corresponde a similitud coseno y cada posición del índice se enlaza con la metadata del chunk.

4.14 RAG

RAG combina recuperación de evidencia con una etapa de respuesta [7]. Aunque suele asociarse con generación, el alcance del laboratorio usa **recuperación semántica + QA extractivo**: MPNet y FAISS recuperan chunks, y BETO-SQAC extrae una respuesta del texto recuperado. No se utiliza un modelo generativo.

5. Herramientas y tecnologías

Las dependencias se declaran en `requirements.txt`; el procedimiento detallado de instalación y ejecución se mantiene en el README del repositorio.

Cuadro 1: Herramientas empleadas y función dentro del laboratorio.

Herramienta	Uso
Python y Jupyter	Desarrollo, ejecución y conservación de evidencia experimental.
Transformers y PyTorch	Tokenización e inferencia con los modelos BETO de QA.
Sentence Transformers	Embeddings MiniLM y MPNet para recuperación semántica.
PyMuPDF	Extracción de texto y página desde archivos PDF [10].
FAISS	Índice vectorial exacto <code>IndexFlatIP</code> .
pandas y Matplotlib	Tablas, agregados y visualizaciones.
LaTeX	Redacción y maquetación del informe.

6. Desarrollo de Question Answering

6.1 Instalación y configuración

Se utilizó un entorno Python con Jupyter, PyTorch y Transformers. Las dependencias se instalaron desde el archivo del proyecto; los notebooks cargaron el dispositivo disponible y ejecutaron inferencia sin modificar pesos. La evidencia final quedó conservada en el notebook canónico.

6.2 Carga del modelo

La primera fase cargó BETO-SQuAD2 (mrm8488) mediante la clase automática de Transformers para QA. En las fases comparativas se incorporaron otro BETO-SQuAD2 y BETO-SQAC; la selección final se presenta en la sección 9.

6.3 Tokenizer

El tokenizer del mismo checkpoint codifica pregunta y contexto, aplica truncamiento a 512 tokens y devuelve tensores con identificadores y máscara de atención. Mantener tokenizer y modelo del mismo checkpoint evita vocabularios incompatibles.

6.4 Pipeline QA

La interfaz QA produce campos equivalentes a `answer`, `score`, `start` y `end`. Para controlar varios spans, el laboratorio implementó la misma lógica directamente con logits de inicio y fin, en lugar de depender únicamente de una llamada de alto nivel [8].

6.5 Función de respuesta

`responder_pregunta(contexto, pregunta, top_k)` tokeniza la entrada, ejecuta el modelo sin gradientes, combina posiciones válidas de inicio y fin, decodifica los spans y los ordena por score. La función se reutilizó en los contextos breves.

6.6 Interpretación del resultado

`answer` contiene el span extraído; `start` y `end` ubican sus tokens; `score` expresa confianza del extractor. Una salida no prueba corrección: debe contrastarse con la respuesta esperada y con la presencia real de evidencia.

7. Experimentos iniciales

7.1 Contexto del Sistema Solar

Se construyó un contexto controlado sobre formación, planetas, tamaño de Júpiter, Ganímedes y distribución de masa del Sistema Solar. Su brevedad permitió observar directamente cómo el extractor localiza spans.

7.2 Preguntas de prueba

Se formularon tres preguntas: fecha aproximada de formación, planeta de mayor tamaño y satélite natural más grande. Todas tenían una respuesta explícita en el contexto.

7.3 Análisis de respuestas

Cuadro 2: Resultados iniciales con el contexto del Sistema Solar.

Pregunta	Respuesta rank 1	Score	top_k
¿Cuándo se formó?	hace aproximadamente 4.6 mil millones de años	0.4050	3
¿Cuál es el planeta más grande?	Júpiter	0.6399	5
¿Cuál es el satélite natural más grande?	Ganímedes	0.8540	5

Las tres respuestas conservaron el contenido esperado. El primer caso tuvo menor score y variaciones en los límites del span; Ganímedes produjo la separación más clara entre la primera hipótesis y las demás.

7.4 Análisis de score

Los scores variaron de 0.4050 a 0.8540 aun con respuestas correctas. Esto demuestra que el score sirve para ordenar hipótesis del mismo modelo, pero no es una métrica binaria de acierto.

7.5 Uso de top_k

Con valores 1, 3 y 5, la respuesta rank 1 permaneció estable en las tres preguntas. Con `top_k=5`, casi todas las alternativas fueron ampliaciones o recortes del mejor span y su

score descendió. El parámetro resultó útil para inspeccionar ambigüedad, no para mejorar automáticamente la primera respuesta.

7.6 Preguntas cuya respuesta no está en el contexto

Ante «¿Quién inventó Internet?» el modelo devolvió tokens ajenos a una respuesta con score 0.000045; ante «¿Cuál es la capital de Francia?», el máximo fue 0.0040. El extractor siempre intenta seleccionar un span, por lo que una pregunta sin evidencia requiere abstención o validación adicional.

8. Actividad de laboratorio: contexto propio

8.1 Descripción del contexto seleccionado

Se eligió Inteligencia Artificial y Machine Learning en sistemas informáticos. El texto reúne aprendizaje supervisado y no supervisado, ciclo de desarrollo, despliegue, riesgos y criterios de uso responsable.

8.2 Contexto de aproximadamente 300 palabras

En los sistemas informáticos modernos, la inteligencia artificial (IA) reúne métodos que permiten automatizar tareas como clasificar mensajes, reconocer imágenes y apoyar decisiones. Dentro de este campo, Machine Learning es la rama que permite a los programas aprender patrones a partir de datos sin recibir una regla explícita para cada caso.

El aprendizaje supervisado utiliza ejemplos etiquetados: cada entrada se acompaña de la salida correcta. Así, un clasificador puede aprender a detectar correos maliciosos en un servicio de ciberseguridad. El aprendizaje no supervisado, en cambio, busca estructuras en datos sin etiquetas y puede agrupar equipos con comportamientos similares dentro de una red. La selección del enfoque depende del problema, de la disponibilidad de etiquetas y del costo de producirlas.

Integrar un modelo en un sistema informático exige más que entrenarlo. Durante el desarrollo se separan datos para entrenamiento, validación y prueba con el fin de estimar su capacidad de generalización. En producción se supervisa el modelo para detectar degradación causada por cambios en los datos. Cuando la latencia y la privacidad son prioritarias, la inferencia puede ejecutarse en dispositivos periféricos mediante edge computing; en otros casos, se emplean servicios en la nube para escalar el procesamiento.

También existen riesgos. Si los datos históricos representan de forma desigual a distintos grupos, puede aparecer sesgo y producir decisiones injustas. La trazabilidad de los datos, el código y los parámetros permite reconstruir qué versión del modelo produjo un resultado. La explicabilidad ayuda a comunicar los factores que influyeron en una predicción, mientras que la supervisión humana permite revisar casos sensibles. Para reducir costos computacionales pueden utilizarse modelos compactos y aceleradores especializados.

Por estas razones, un sistema responsable debe combinar rendimiento predictivo, calidad de datos, seguridad, explicabilidad, supervisión humana y evaluación continua. Machine Learning aporta valor cuando el objetivo está bien definido y sus resultados se verifican con criterios técnicos y éticos.

El recuento ejecutado fue de 308 palabras.

8.3 Diseño de las 10 preguntas

Se definieron tres preguntas fáciles, cuatro intermedias, dos de relación y una difícil. Cubrieron definiciones, etapas del desarrollo, ciberseguridad, despliegue, sesgo, trazabilidad, supervisión, recursos y responsabilidad. Las diez respuestas esperadas aparecen en el contexto; el detalle se conserva en el anexo 18.1.

8.4 Resultados obtenidos

Con BETO-SQuAD2 y `top_k=1`, la evaluación normalizada más la revisión manual explícita de la pregunta 5 registró 10/10. El score medio fue 0.6316 y el tiempo total 1.7577 s. Este resumen corresponde al protocolo `qa_basico`; no debe confundirse con el benchmark histórico de la sección 9.

8.5 Tabla de respuestas y scores

Cuadro 3: Respuestas del contexto propio.

#	Respuesta extraída	Score	Correcta
1	machine learning	0.8749	Sí
2	entrenamiento, validación y prueba	0.6376	Sí
3	aprendizaje supervisado	0.7874	Sí
4	correos maliciosos	0.9291	Sí
5	en dispositivos periféricos	0.2518	Sí*
6	sesgo y producir decisiones injustas	0.5374	Sí
7	trazabilidad de los datos, el código y los parámetros	0.4900	Sí
8	detectar degradación causada por cambios en los datos	0.3547	Sí
9	modelos compactos y aceleradores especializados	0.7412	Sí
10	rendimiento predictivo, calidad de datos, seguridad, explicabilidad, supervisión humana y evaluación continua	0.7116	Sí

*Respuesta suficiente al lugar solicitado, pero parcial: omitió «mediante edge computing»; se registró revisión manual.

8.6 Análisis de errores

No hubo errores finales bajo el criterio declarado, pero sí límites imperfectos. La pregunta 5 produjo una respuesta parcial y score bajo; la 6 añadió texto sin perder el concepto central. Los casos muestran que la exactitud normalizada y el score describen aspectos distintos.

8.7 Reflexión

El modelo funcionó bien cuando la evidencia era explícita y el contexto compacto. `top_k` ayuda a revisar alternativas y el score ayuda a priorizarlas, pero ninguno sustituye la evaluación del contenido. Un extractor no puede aportar conocimiento ausente del contexto.

9. Comparación de modelos QA

9.1 Modelos utilizados

Se compararon tres checkpoints, abreviados como BETO-SQuAD2 (mrm8488), BETO-SQuAD2 (MMG) y BETO-SQAC (MMG). Los identificadores ejecutados se conservan en el notebook; la configuración final exacta se presenta en la sección 14.10.

9.2 Contexto de evaluación

La comparación controlada final usó tres contextos: introducción institucional de la UES, bases de datos/SGBD y un artículo sobre aprendizaje automático. Cada modelo respondió cinco preguntas por contexto, para 15 inferencias.

9.3 Preguntas

Las preguntas exigieron fechas, definiciones, relaciones, listados y descripciones explícitas. Se mantuvieron iguales entre modelos; las diez preguntas de bases de datos y Wikipedia se presentan en la sección 10.

9.4 Resultados por modelo

Cuadro 4: Comparación controlada final: 15 preguntas por modelo.

Modelo	Correctas	Accuracy	Score medio	Tiempo medio (s)
BETO-SQuAD2 (mrm8488)	13/15	86.67 %	0.5412	0.1215
BETO-SQuAD2 (MMG)	13/15	86.67 %	0.4249	0.1132
BETO-SQAC (MMG)	15/15	100.00 %	0.6920	0.1154

9.5 Score

BETO-SQAC obtuvo el mayor score medio (0.6920), además del mayor número de aciertos. El score no se usó por sí solo para seleccionar el modelo: se contrastó con respuestas esperadas y revisión de spans.

9.6 Tiempo de inferencia

Los promedios fueron cercanos, entre 0.1132 y 0.1215 s. Son mediciones de una ejecución concreta y no un benchmark universal; otras tablas históricas se ejecutaron bajo condiciones distintas.

9.7 Ventajas y desventajas

BETO-SQAC presentó la mejor exactitud y confianza media, por lo que fue elegido como extractor final. BETO-SQuAD2 (MMG) fue ligeramente más rápido en este registro, pero falló dos casos; mrm8488 facilitó la fase introductoria y tuvo score intermedio, también con dos fallos.

9.8 Análisis comparativo

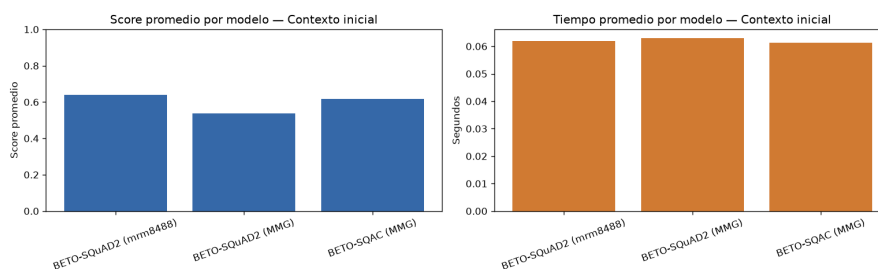


Figura 1: Comparación de modelos registrada en la fase histórica. La figura apoya la tendencia de score y tiempo; la decisión final usa la tabla controlada de 45 inferencias.

`comparacion_modelos.csv` conserva un benchmark histórico de 60 inferencias (20 por modelo), mientras `resultados_qa.csv` contiene la comparación controlada final de 45. Sus agregados no se mezclaron; el anexo 18.3 documenta ambos protocolos.

9.9 Conclusiones

BETO-SQAC (MMG) fue el modelo más adecuado para las preguntas controladas: 15/15, score medio 0.6920 y tiempo semejante a los otros checkpoints. Por ello se mantuvo fijo en los experimentos documentales posteriores.

10. Experimentos con diferentes contextos

10.1 Bases de Datos / IA / Redes Neuronales / Ciencia de Datos

El contexto especializado implementado fue Bases de Datos y SGBD. Explicó definición de gestor, modelo relacional, claves primaria y foránea y propiedades ACID. Los demás temas se abordaron dentro del contexto propio de IA y del artículo de aprendizaje automático, sin inventar un corpus adicional.

10.2 Resultados

Sobre cinco preguntas de bases de datos, mrm8488 y BETO-SQuAD2 (MMG) respondieron 4/5, mientras BETO-SQAC respondió 5/5. Sus scores medios fueron 0.4367, 0.3297 y 0.7785, respectivamente.

10.3 Artículo de Wikipedia

Se utilizó un fragmento de Wikipedia sobre aprendizaje automático, tipos de aprendizaje y uso de experiencia o retroalimentación. El texto se trató como un contexto separado y común a los tres modelos.

10.4 Cinco preguntas

Se preguntó: (1) objetivo del aprendizaje automático; (2) cuándo se considera que un agente aprende; (3) qué correspondencia establece el aprendizaje supervisado; (4) qué reconoce el no supervisado; y (5) qué información usa el aprendizaje por refuerzo.

10.5 Comparación de los tres modelos

Cuadro 5: Resultados controlados por contexto (cinco preguntas por celda).

Modelo	Correctas / score medio		
	UES	Bases de datos	Wikipedia
BETO-SQuAD2 (mrm8488)	4/5 / 0.6400	4/5 / 0.4367	5/5 / 0.5469
BETO-SQuAD2 (MMG)	5/5 / 0.5369	4/5 / 0.3297	4/5 / 0.4082
BETO-SQAC (MMG)	5/5 / 0.6158	5/5 / 0.7785	5/5 / 0.6816

El resultado por dominio confirma la mayor consistencia de BETO-SQAC. No se deduce superioridad general fuera de estos tres contextos y quince preguntas.

11. Question Answering sobre documentos extensos

11.1 Problema de longitud de los Transformers

El documento fuente tiene 62 páginas y 26 119 palabras. No cabe en los 512 tokens admitidos por el extractor, de modo que truncarlo eliminaría la mayor parte de la evidencia.

11.2 Estrategia de chunking

Se extrajo texto página por página, se normalizaron espacios y saltos sin perder la procedencia y se generaron ventanas de caracteres. Cada fragmento conservó identificador, texto y páginas de inicio/fin.

11.3 Tamaño de fragmentos

Se evaluaron 300, 500 y 800 caracteres, con y sin overlap cuando existía un registro real. Los tamaños produjeron entre 217 y 694 chunks. La comparación completa se presenta en las secciones 12 y 15.

11.4 Implementación

Para el barrido inicial, BETO-SQAC procesó todos los chunks de cada pregunta. Luego se ordenaron las respuestas por score QA. Esta línea base permitió medir la fragmentación antes de incorporar embeddings.

11.5 Selección de la mejor respuesta

La respuesta ganadora fue el span con mayor score entre todos los fragmentos. Además se verificó si el chunk contenía la evidencia esperada. Esta separación evitó considerar correcto un span de alta confianza extraído de contexto irrelevante.

11.6 Análisis de resultados

Sin recuperación semántica, 300/0 y 500/0 obtuvieron 3/10; 800/0 obtuvo 4/10. El score medio alto (0.9172–0.9428) coexistió con baja exactitud, señal de que comparar scores entre numerosos chunks favorece falsos positivos. La configuración 800/0 redujo el barrido a 217 chunks y se conservó para el RAG.

12. Evaluación del overlap

12.1 Chunking sin overlap

Las configuraciones 300/0, 500/0 y 800/0 produjeron 579, 347 y 217 chunks, con 3/10, 3/10 y 4/10 respuestas correctas.

12.2 Chunking con overlap

Se probaron 300/50, 500/100 y 800/150. Generaron 694, 434 y 267 chunks, respectivamente, y las tres alcanzaron 4/10.

12.3 Resultados

La tabla siguiente reúne los seis registros y permite comparar aciertos, volumen de fragmentos y tiempo total.

Cuadro 6: Seis configuraciones reales de chunking y overlap.

Chunk	Overlap	Chunks	Correctas	Tiempo total (s)
300	0	579	3/10	288.147
300	50	694	4/10	350.798
500	0	347	3/10	223.805
500	100	434	4/10	325.274
800	0	217	4/10	236.143
800	150	267	4/10	282.039

12.4 Comparación

El overlap elevó el acierto de 3/10 a 4/10 para tamaños 300 y 500, pero no mejoró 800. En todos los pares aumentó número de chunks y tiempo. Los tiempos describen esas ejecuciones y no aíslan todos los factores de hardware.

12.5 Conclusiones

El overlap ayudó cuando protegió evidencia cercana a cortes, pero su beneficio no fue consistente. Se eligió 800/0 porque igualó el máximo observado con menos chunks, sin afirmar que overlap cero sea óptimo para cualquier corpus.

13. Sistema RAG

13.1 Arquitectura propuesta

El sistema separa una fase de indexación, ejecutada al preparar el corpus, y una fase de consulta, ejecutada por pregunta. La figura 2 muestra el flujo sin depender del código.

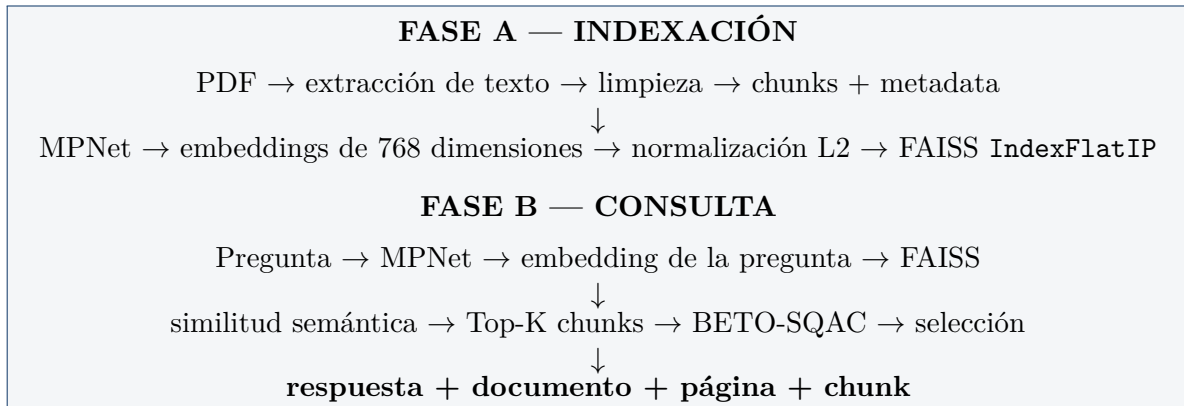


Figura 2: Arquitectura conceptual del RAG extractivo.

13.2 Generación de embeddings

Durante la indexación, cada chunk se codifica una vez. Durante la consulta se codifica la pregunta con el mismo modelo. Los vectores se normalizan para que el producto interno tenga interpretación de similitud coseno.

13.3 Modelo de embeddings

La configuración final usa MPNet multilingüe de Sentence Transformers, con 768 dimensiones [9]. Su identificador completo aparece en la tabla 8. MiniLM de 384 dimensiones se conserva solo como alternativa experimental en la sección 15.1.

13.4 Construcción de la base vectorial

Los embeddings forman una matriz cuya fila i corresponde al registro i de metadata. Esta alineación permite traducir una posición recuperada en texto, documento, página y chunk. En el caso base la matriz contiene 217 filas.

13.5 FAISS

IndexFlatIP agrega los vectores normalizados y efectúa búsqueda exacta por producto interno. No entrena centroides ni aproxima vecinos; para las escalas del laboratorio ofreció un comportamiento simple y verificable.

13.6 Búsqueda semántica

FAISS devuelve posiciones y valores de similitud. El **score de similitud** mide cercanía entre pregunta y chunk. No debe confundirse con el **score QA**, que BETO calcula después para un span dentro del chunk. Un chunk puede ser semánticamente cercano y aun producir una respuesta incorrecta.

13.7 Recuperación Top-K

Se evaluaron $K = 1, 3, 5, 10$ con MiniLM. Recall@K y QA Accuracy pasaron de 10 % en Top-1 a 40 % en Top-10; el tiempo total pasó de 0.189 a 1.408 s para diez preguntas. Top-10 se eligió por la mayor cobertura observada.

13.8 Question Answering sobre fragmentos recuperados

BETO-SQAC recibe únicamente los diez chunks recuperados, extrae un candidato por fragmento y selecciona el mayor score QA. La ejecución validada ofrece un ejemplo: para la periodicidad del Consejo Académico devolvió «una vez al mes», con score QA 0.983679 y similitud 0.639237. Los dos números proceden de etapas distintas.

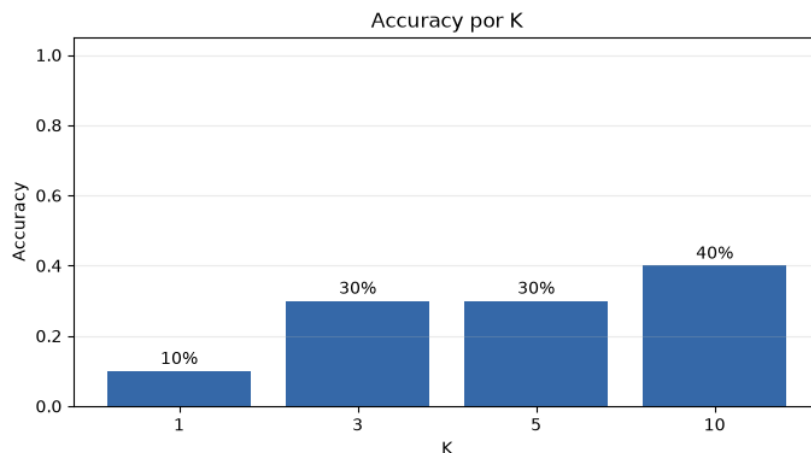


Figura 3: Efecto de Top-K en la exactitud del RAG histórico con MiniLM.

13.9 Extensión multidocumento

El mismo pipeline se generalizó a nueve PDF individuales. Los documentos se procesan por separado y convergen en una base común:

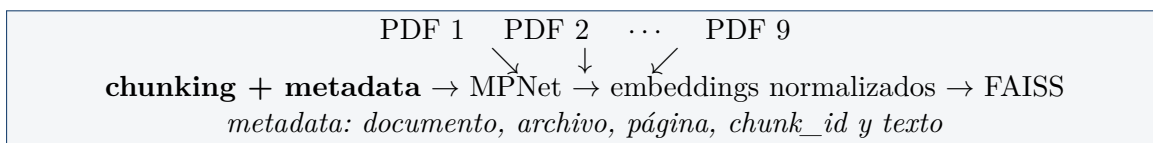


Figura 4: Generalización del índice a múltiples fuentes.

El corpus reunió 9 documentos, 199 páginas y 765 chunks. La metadata permite devolver la procedencia y evaluar por separado el documento y el fragmento. Sobre 16 preguntas respondibles y 2 sin respuesta se obtuvieron:

Cuadro 7: Métricas oficiales del RAG multidocumento.

Métrica	Resultado
Document Recall@10	93.75 %
Chunk Recall@10	56.25 %
MRR documento	0.7833
MRR chunk	0.31875
QA Accuracy respondibles	25 % (4/16)
Abstención correcta	0 % (0/2)

El recuperador suele ubicar el documento relevante, pero pierde cobertura al buscar el chunk exacto y vuelve a perder exactitud durante QA. Un caso correcto recuperó la evidencia sobre autonomía y extrajo «en lo docente, lo administrativo y lo económico». En otro, el documento correcto sobre cambio de carrera fue Top-1, pero el chunk fue incorrecto; en un tercero, se recuperó la evidencia de veinte horas por unidad valorativa y BETO extrajo solo

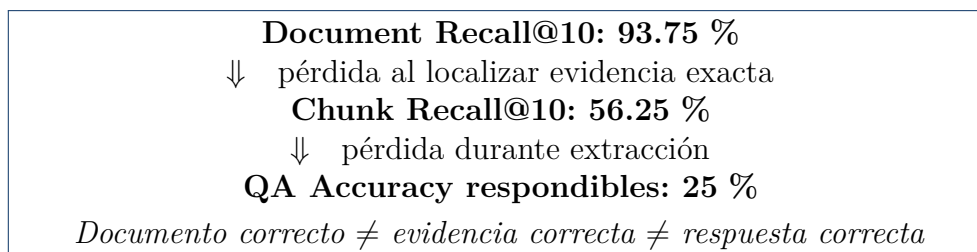


Figura 5: Pérdida observada entre las etapas del sistema multidocumento.

«veinte». La pregunta sobre la contraseña de la red Wi-Fi no tenía evidencia y produjo un span: el mecanismo de abstención no está calibrado.

14. Proyecto final

14.1 Descripción del problema

La normativa universitaria es extensa y distribuye respuestas entre artículos y páginas. El proyecto construyó un asistente que recibe una pregunta, reduce el espacio de búsqueda a fragmentos pertinentes y entrega una respuesta extractiva acompañada de fuente.

14.2 Documento seleccionado

El caso base utilizó el *Reglamento de la Gestión Académico-Administrativa de la Universidad de El Salvador* [11], conservado como `documento_fuente.pdf`: 62 páginas y 217 chunks. La extensión de la sección 13.9 añadió ocho reglamentos individuales, para un total de 9 documentos, 199 páginas y 765 chunks.

14.3 Arquitectura del sistema

Se aplicaron las dos fases de la figura 2. La indexación prepara corpus y metadata; la consulta encadena MPNet, FAISS, Top-10 y BETO-SQAC. No se añadió generación de texto.

14.4 Procesamiento del documento

PyMuPDF extrajo texto de las 62 páginas sin OCR. La limpieza normalizó espacios y preservó saltos útiles. Se fragmentó por caracteres con tamaño 800 y overlap 0, registrando página y posición de cada chunk.

14.5 Embeddings

MPNet produjo una matriz de 217×768 en el caso base. Como se explicó en la sección 13.2, tanto chunks como preguntas se normalizaron antes de la búsqueda.

14.6 FAISS

La matriz se añadió a `IndexFlatIP`. Cada índice entero se mantuvo alineado con su meta-data, por lo que la respuesta puede citar el origen sin buscar nuevamente en el PDF.

14.7 Recuperación semántica

La pregunta se codifica y FAISS devuelve diez chunks con sus similitudes. El `Recall@10` monodocumento indica en qué proporción de diez preguntas la evidencia esperada apareció entre esos diez candidatos.

14.8 QA

BETO-SQAC evalúa los diez fragmentos recuperados y selecciona el span con mayor score QA. La Accuracy mide respuestas finalmente correctas, no cobertura del recuperador. Esta distinción explica por qué ambas métricas deben registrarse.

14.9 Preguntas de prueba

Se usaron diez preguntas con evidencia validada: tres fáciles, cuatro intermedias, dos difíciles y una diseñada para un límite de fragmento. Trataron periodicidad, cambio de carrera, calidad de egresado, unidades administrativas, pruebas de ingreso, asesoría por CUM, nombramientos y vigencia. El anexo 18.2 lista las diez.

14.10 Resultados

Cuadro 8: Configuración y resultado monodocumento final.

Componente	Configuración / resultado
QA	MMG/bert-base-spanish-wwm-cased-finetuned-sqac
Embeddings	paraphrase-multilingual-mpnet-base-v2; 768 dimensiones
Fragmentación	800 caracteres; overlap 0; 217 chunks
Recuperación	FAISS <code>IndexFlatIP</code> ; Top-K 10
Evaluación	<code>Recall@10</code> = 60 %; QA Accuracy = 60 % (6/10)

`Recall@10` = 60 % significa que la evidencia esperada apareció entre los diez recuperados para 6 de 10 preguntas. QA Accuracy = 60 % significa que la respuesta final fue aceptada en 6 de 10. El conjunto es pequeño, de modo que los valores describen esta prueba y no una tasa general de uso.

14.11 Análisis de errores

Los errores se originaron en recuperación, límites de chunk y selección de span. Un score QA elevado en contexto incorrecto no rescata una consulta. La extensión multidocumento hizo

visible el encadenamiento de la figura 5: documento, evidencia y respuesta son condiciones sucesivas, no equivalentes.

14.12 Limitaciones

El conjunto monodocumento contiene solo diez preguntas y el multidocumento, dieciocho. No se calibró un umbral de abstención: las dos preguntas sin respuesta produjeron spans. La fragmentación por caracteres puede separar unidades semánticas y el índice exacto no aplica reranking especializado. Los tiempos proceden de ejecuciones concretas y algunas fases usaron dispositivos distintos.

14.13 Mejoras propuestas

Las mejoras directamente motivadas por los errores son: recuperar mejor el chunk exacto una vez localizado el documento, calibrar abstención con ejemplos sin respuesta y evaluar reranking antes de QA. Deben medirse con un conjunto mayor y mantenido fijo, sin asumir de antemano que elevarán la exactitud.

15. Experimentos adicionales

15.1 Modelo de embeddings alternativo

Se comparó MiniLM multilingüe (384 dimensiones) con MPNet multilingüe (768). Con 800/0 y Top-10, MiniLM obtuvo Recall y Accuracy de 40 %; MPNet alcanzó 60 % en ambas. La generación de embeddings pasó de 0.536 a 1.642 s; el tiempo de consulta fue cercano (1.432 frente a 1.420 s).

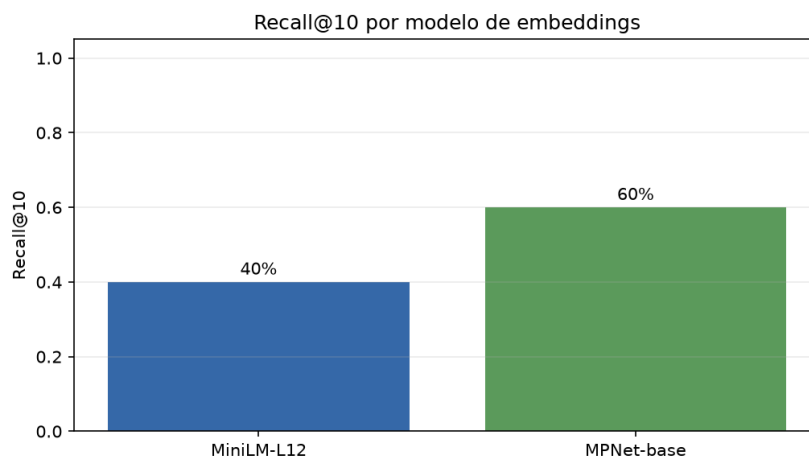


Figura 6: Recall@10 registrado para MiniLM y MPNet.

15.2 Tamaño de chunk: 300, 500 y 800

Sin overlap, 300 y 500 caracteres obtuvieron 3/10, mientras 800 obtuvo 4/10. También redujo el número de chunks de 579 y 347 a 217. La selección de 800 se basó en el mejor resultado observado con menor base, no en una regla universal.

15.3 Diferentes valores de overlap

Se evaluaron los pares reales 300/0–300/50, 500/0–500/100 y 800/0–800/150. El overlap mejoró un acierto en los dos primeros tamaños y no cambió el tercero; siempre aumentó chunks y tiempo. Por eso la configuración final mantuvo overlap 0.

15.4 Diferentes valores de k

Cuadro 9: Top-K con MiniLM, 800/0 y diez preguntas.

K	Recall@K	Accuracy	Score QA medio	Tiempo total (s)
1	10 %	10 %	0.2694	0.189
3	30 %	30 %	0.6407	0.508
5	30 %	30 %	0.7027	0.721
10	40 %	40 %	0.8364	1.408

Top-10 ofreció la mayor cobertura del experimento, con mayor coste que Top-1. Después se mantuvo fijo al comparar embeddings.

15.5 Impacto del tamaño de la base vectorial

Cuadro 10: Tamaño de base con MPNet, 800/0 y Top-10.

Vectores	Recall@10	Accuracy	Tiempo total (s)
50	70 %	50 %	1.408
100	70 %	60 %	1.410
150	60 %	60 %	1.403
217	60 %	60 %	1.412

Al agregar distractores, Recall descendió de 70 % a 60 %, mientras el tiempo permaneció cercano con Top-10 fijo. Las bases pequeñas fueron construcciones controladas que preservaron evidencia, por lo que el resultado describe el efecto de distractores en este conjunto.

16. Conclusiones

1. **Resultados principales.** El flujo monodocumento final obtuvo Recall@10 y QA Accuracy de 60 % (6/10). En multidocumento, Document Recall@10 fue 93.75 %, Chunk

Recall@10 56.25 % y QA Accuracy responsable 25 % (4/16); las dos abstenciones fallaron.

2. **Modelo QA más adecuado.** BETO-SQAC (MMG) logró 15/15 en la comparación controlada, frente a 13/15 de cada BETO-SQuAD2, con tiempo medio semejante; por ello fue el extractor final.
3. **Efecto del chunking.** Entre los tamaños sin overlap, 800 caracteres alcanzó 4/10 con solo 217 chunks, frente a 3/10 con 300 y 500.
4. **Efecto del overlap.** El solapamiento ayudó en dos pares, pero no mejoró 800/0 y siempre elevó volumen y tiempo; el resultado no respalda asumir que más overlap siempre es mejor.
5. **Efectividad de la búsqueda semántica.** Top-10 amplió la cobertura histórica de MiniLM a 40 %, y MPNet la elevó a 60 % en el caso base. En múltiples documentos localizó con frecuencia la fuente correcta, pero no siempre el chunk exacto.
6. **Ventajas de RAG.** La recuperación evita evaluar todo el documento por consulta, permite escalar a varias fuentes y conserva documento, página y chunk para sustentar la respuesta.
7. **Limitaciones.** Documento correcto no implica evidencia correcta, ni evidencia correcta implica respuesta correcta. La brecha entre 93.75 %, 56.25 % y 25 % identifica pérdidas sucesivas; además, 0/2 abstenciones correctas evidencia la necesidad de calibrar preguntas sin respuesta.

17. Referencias bibliográficas

- [1] A. Vaswani et al., «Attention Is All You Need», *NeurIPS*, 2017. <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
- [2] J. Devlin, M.-W. Chang, K. Lee y K. Toutanova, «BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding», *NAACL-HLT*, 2019. <https://aclanthology.org/N19-1423/>
- [3] J. Cañete et al., «Spanish Pre-Trained BERT Model and Evaluation Data», *PML4DC, ICLR*, 2020. https://pml4dc.github.io/iclr2020/program/pml4dc_10.html
- [4] N. Reimers e I. Gurevych, «Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks», *EMNLP-IJCNLP*, 2019. <https://aclanthology.org/D19-1410/>
- [5] K. Song et al., «MPNet: Masked and Permuted Pre-training for Language Understanding», *NeurIPS*, 2020. [Fuente oficial en NeurIPS](#).
- [6] J. Johnson, M. Douze y H. Jégou, «Billion-scale similarity search with GPUs», *IEEE Transactions on Big Data*, 2019. <https://arxiv.org/abs/1702.08734>
- [7] P. Lewis et al., «Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks», *NeurIPS*, 2020. <https://proceedings.neurips.cc/paper/2020/hash/6b493230-Abstract.html>

- [8] Hugging Face, «Question answering», documentación oficial de Transformers. https://huggingface.co/docs/transformers/tasks/question_answering
- [9] Sentence Transformers, «paraphrase-multilingual-mpnet-base-v2», ficha del modelo. <https://huggingface.co/sentence-transformers/paraphrase-multilingual-mpnet-base-v2>
- [10] Artifex Software, «PyMuPDF: The Basics», documentación oficial. <https://pymupdf.readthedocs.io/en/latest/the-basics.html>
- [11] Universidad de El Salvador, *Reglamento de la Gestión Académico-Administrativa de la Universidad de El Salvador*, 2013, con modificaciones consignadas hasta 2017.

18. Anexos

18.1 Preguntas del contexto propio

1. ¿Qué rama permite a los programas aprender patrones a partir de datos?
2. ¿Para qué tres etapas se separan los datos durante el desarrollo?
3. ¿Qué tipo de aprendizaje utiliza ejemplos etiquetados?
4. ¿Qué puede aprender a detectar un clasificador en ciberseguridad?
5. ¿Dónde puede ejecutarse la inferencia cuando la latencia y la privacidad son prioritarias?
6. ¿Qué puede aparecer si los datos históricos representan de forma desigual a distintos grupos?
7. ¿Qué permite reconstruir qué versión del modelo produjo un resultado?
8. ¿Para qué se supervisa el modelo en producción?
9. ¿Qué recursos pueden utilizarse para reducir costos computacionales?
10. ¿Qué elementos debe combinar un sistema responsable?

18.2 Preguntas monodocumento

1. ¿Con qué periodicidad sesiona ordinariamente el Consejo Académico?
2. ¿Cuál es el máximo de veces que un estudiante puede cambiar de carrera?
3. ¿Cuánto dura ordinariamente la calidad de egresado?
4. ¿Qué unidades integran internamente la Secretaría de Asuntos Académicos?
5. ¿Qué prueba se aplica después del curso de refuerzo académico en línea?
6. ¿A qué proceso se somete al estudiante con CUM acumulado menor a siete punto cero?
7. ¿Quién nombra a los miembros del Tribunal Calificador?
8. ¿Según qué reglamento se tramita la selección de ingreso de un aspirante a posgrado?
9. ¿Cuándo entra en vigencia el Reglamento?
10. ¿Por cuánto tiempo se amplía automáticamente la calidad de egresado si el plazo vence después de aprobar el trabajo de graduación y antes del acto de graduación?

18.3 Protocolos de comparación de modelos

Cuadro 11: Separación de los dos artefactos comparativos.

Fuente	Protocolo	mrm8488	MMG SQuAD2	MMG SQAC
comparacion_modelos.csv	Histórico: 20 preguntas por modelo	17/20	16/20	18/20
resultados_qa.csv	Control final: 15 preguntas por modelo	13/15	13/15	15/15

El contexto propio aparece en ambos recorridos con criterios registrados en cada artefacto. Los resultados del cuerpo usan el resumen `qa_basico` para la actividad y las 45 inferencias controladas para seleccionar el modelo.

18.4 Comparación descriptiva mono y multidocumento

Cuadro 12: Escala y configuración de los dos sistemas.

Característica	Monodocumento	Multidocumento
Documentos	1	9
Páginas	62	199
Chunks	217	765
Embedding	MPNet	MPNet
Dimensión	768	768
Top-K	10	10
QA	BETO-SQAC	BETO-SQAC

El Recall@10 monodocumento verifica la presencia de la evidencia esperada entre chunks del único documento. Document Recall@10 multidocumento verifica si aparece el documento esperado. No son métricas intercambiables; por eso la extensión reporta además Chunk Recall@10.

18.5 Evidencia y repositorio

El notebook canónico es `LaboratorioML.ipynb`; la demostración ejecutada del flujo final es `RAG_UES_Demo.ipynb`. Los resultados estructurados se encuentran en `results/`, y el código de apoyo en `src/`. El repositorio está disponible en: <https://github.com/abner-rivas/Laboratorio---ML.git>.